Team Name : puranpoli

Team Leader Name : Ram Madanrao Kandalkar

Problem Statement : Data & AI Challenge : Intelligent Candidate Discovery

Solution Overview

What we built:

- A Candidate Intelligence Ranker — not a keyword filter, but a learning-to-rank pipeline that scores 100,000 profiles against a JD and surfaces the top 100 best-fit candidates with evidence-backed reasoning.

What makes it different:

- Evidence over keywords → we detect production ownership of ranking/retrieval/recommendation systems, not just skill mentions
- Learned ranking → pairwise reranker trained on 520 human-reviewed labels to learn what "good fit" actually means
- Fake-risk defense → inflated, inconsistent, or hollow profiles are actively penalized
- Zero hallucination → no live LLM at ranking time; fully deterministic and CPU-only

JD Understanding & Candidate Evaluation

Must-have fit

ML/search/recommendation background, 5-9 years, production ownership.

High-value proof

Ranking/retrieval systems, NDCG/MRR, A/B tests, offline-online evaluation.

Risk checks

Mismatched experience, keyword stuffing, stale profiles, suspicious timelines.

How we evaluate fit -> beyond keywords:

Tier	What they have
🟢 Strong	Owned real relevance/ranking systems end-to-end
🟡 Medium	Adjacent ML/RAG/semantic-search, limited ranking ownership
🔴 Weak	ML vocabulary, no production search/recs evidence

Cross-checks applied:

- Career history consistency
- Platform signals (verified contact, response rate, recent activity)
- Risk flags (mismatched years, suspicious timelines, domain mismatch)

Ranking Methodology

1. Feature extraction

Turn each profile into fit, skill, evidence, logistics, and risk features

2. Reviewed labels

Use 520 reviewed labels to teach what good ranking looks like.

3. Pairwise reranking

Learn candidate A should rank above candidate B when label A is better.

4. Final sort

Blend learned score with rule score and produce top 100..

How the reranker thinks:

Label 5 candidate > Label 3 candidate > Label 1 candidate

Pairwise logistic loss rank order matters, not arbitrary numbers.

Runtime constraints fully met:

- CPU-only, fully offline
- Memory-safe streaming (465MB file, never fully loaded)
- No GPU · No paid API · No cloud dependency

Explainability & Data Validation

How we explain every ranking decision:

- Deterministic, template-based reasoning — generated from the candidate's actual fields
- Years · Title · Company · Evidence terms · Location · Logistics signals
- Example output: "7.9 yrs Senior AI Engineer at Meta — detected: ranking/retrieval systems, offline metrics, semantic search"

Hard QA rules enforced on final top 100:

- Fake-risk > 0.10 → 0 candidates
- Under 5 years experience → 0 candidates

Zero hallucination by design:

No LLM is called during ranking or explanation.
Every sentence traces back to a feature **actually detected** in the profile.

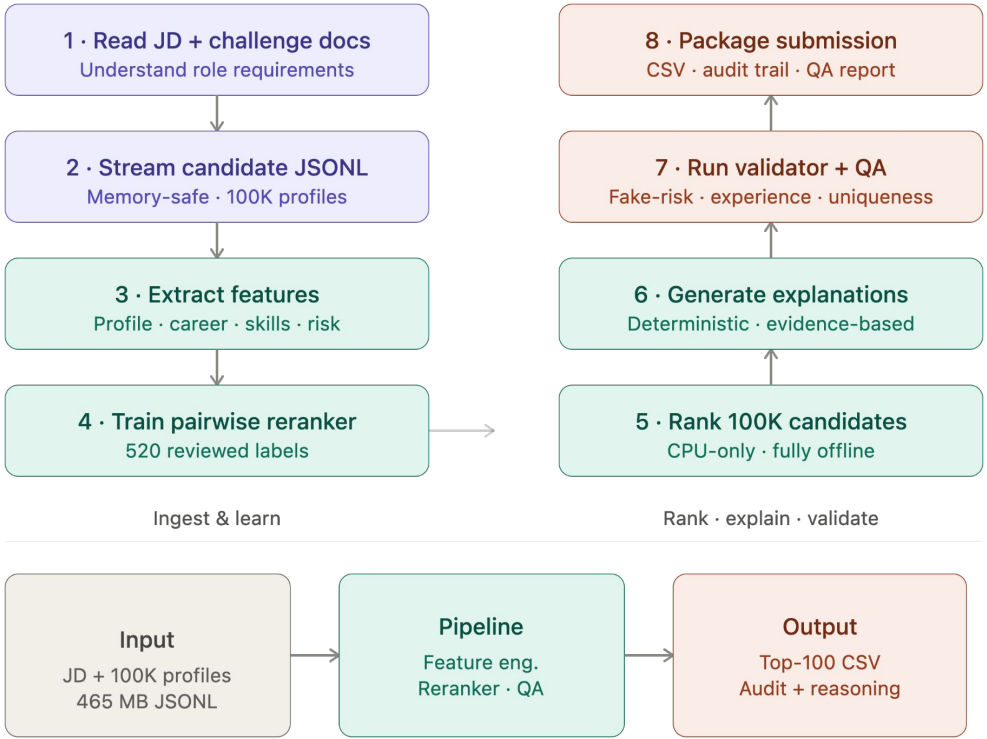
How we catch bad profiles:

Flag	What triggers it
Mismatched years	Stated experience ≠ career timeline
Suspicious timeline	Overlapping or impossible job dates
Domain mismatch	Claims don't align with actual work history
Keyword stuffing	Skill-rich but zero production evidence

End-to-End Workflow

End-to-End Workflow

JD input → ranked shortlist in 8 steps.
No live LLM.
No GPU.
No cloud dependency.

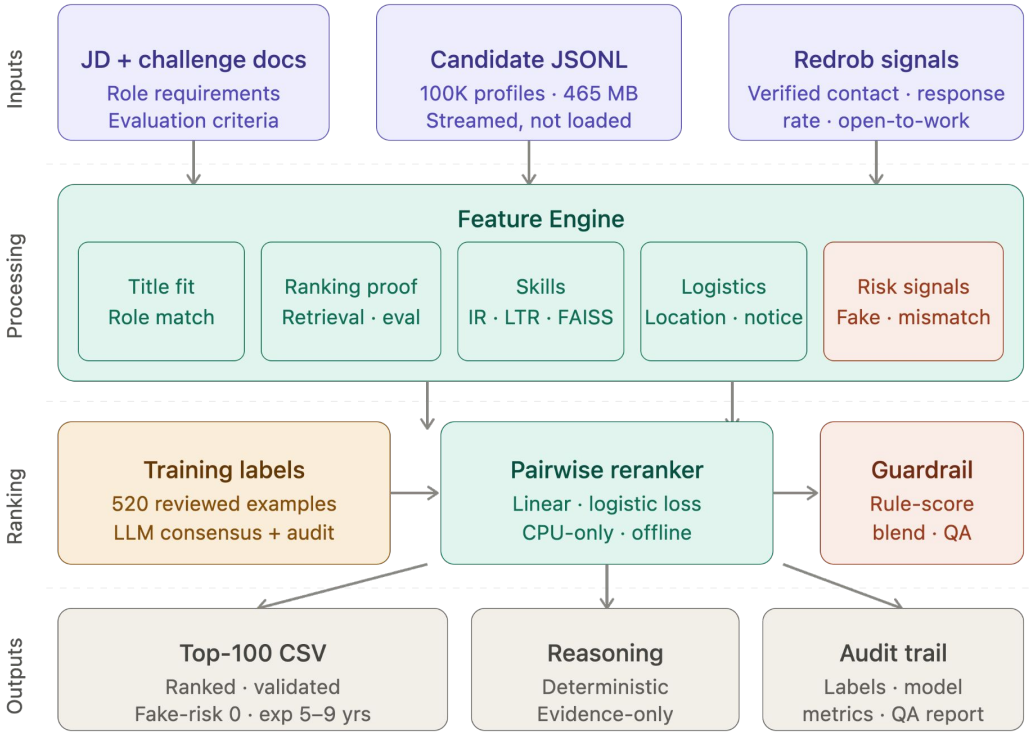


System Architecture

Key design decisions:

- No external inference service — everything runs on local CPU
- Streaming architecture handles 465MB without memory overload
- Guardrail layer prevents the learned model from drifting off JD logic
- Risk signals are a dedicated layer, not an afterthought
- Every output node is fully auditable — judges can trace any ranking back to raw features

"No black boxes. Every score has a source."



Results & Performance

Validator Passed ✓	Rows 100	Fake-risk leaks 0
Under-5 leaks 0	Pair accuracy 0.961	MAP label ≥ 3 0.976

Technologies Used

- Python standard library for streaming JSONL, scoring, CSV writing, and offline ranking.
- Custom feature extraction for ranking/retrieval/evaluation/production evidence.
- Pairwise linear reranker implemented without heavy ML dependencies.
- ReportLab for generated PDFs and documentation assets.
- pdfplumber/pypdf for PDF extraction and QA.
- No live LLM, GPU, cloud database, or paid API needed at submission time.

Submission Assets

- final/team_redrob_ranker_v2_final.csv - final upload CSV.
- final/audit_top300_v2_final.csv - evidence and feature audit.
- final/qa_report_final.json - validator and quality checks.
- all_csv/ - labels, predictions, audits, and variant outputs.
- all_json/ - models, metrics, and reports.
- code/ - complete scripts used to train, rank, validate, and document.
- docs/ - approach PDF, PPT-content PDF, workflow notes, and implementation report.

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU